

Exercise-1

```
© Singleton.java × TestSingleton.java
1 package DN5.SingletonPattern;
2 public class Singleton {
3     private static Singleton instance;
4     private Singleton() {
5         System.out.println("Singleton instance created");
6     }
7     public static Singleton getInstance() {
8         if(instance == null){
9             instance = new Singleton();
10        }
11        return instance;
12    }
13    public void log(String s){
14        System.out.println(s);
15    }
16 }
```

```
© Singleton.java TestSingleton.java ×
1 package DN5.SingletonPattern;
2
3 public class TestSingleton {
4     public static void main(String[] args) {
5
6         Singleton s1 = Singleton.getInstance();
7         Singleton s2 = Singleton.getInstance();
8
9         s1.log("Application Created");
10        s2.log("User Logged in");
11
12        System.out.println(s1==s2);
13    }
14
15 }
```

Output

```
Run TestSingleton x
"C:\Program Files\Java\jdk-24\bin\java.exe" --en
Singleton instance created
Application Created
User Logged in
true

Process finished with exit code 0
```

Theory:-

logging = console.log.

PAGE NO.: _____ DATE ____/____/____

→ Singleton Pattern (Creational):

This pattern ensures:

- Only one object of a class is created.
- A global access point is provided to that object.
- Only one obj. is possible to create not more than one.

Note: Types of memory:

- Stack: Stores primitive data type.
eg. int, char, bool.
- Heap: stores ^{non} primitive data type.
e.g. Classes and Objects.

In case of string its reference or pointer is stored in ~~stack~~ stack but its actual data is stored in heap. It is non-primitive data type.

* Singleton Real world use:

- 1) Logging System
- 2) Database Connection
- 3) Configuration Manager.